# Forja de Blibblon: Representações Esparsas, Gestão de Equipamentos e Recomendação de Itens

**Lucas Siqueira Campo Dall Orto -**

Universidade Federal de Minas Gerais
Ciência da Computação – Estruturas de Dados

**Abstract**

Este trabalho apresenta um sistema de gerenciamento de uma forja, no qual itens modificam poucos atributos, pedidos são atendidos em ordem de chegada e equipamentos são removidos em ordem inversa à aquisição. A implementação em C++ utiliza tipos abstratos de dados próprios, uma fila global, uma pilha por aventureiro e um catálogo esparso com migração entre lista cruzada, tripletas ordenadas e armazenamento comprimido por linha. A recomendação combina ordenações globais por poder, equilíbrio e sinergia; uma extensão permite consultas por faixa, incluindo modificadores zero sem célula armazenada. A avaliação compara perfis de atualizações e consultas, estratégias de manutenção da build, três ordenadores e dois produtos internos. Na configuração base com predominância de consultas, os tempos medianos da consulta de recomendação foram 103,0, 132,4 e 110,7 $\mu$s nos modos L, T e C, respectivamente. A lista cruzada favoreceu os tempos dessas cargas; a representação comprimida reduziu a memória. Os resultados evidenciam que contiguidade, custo de localização e manutenção de índices devem ser avaliados conjuntamente.

## 1 Introdução

Sistemas de equipamentos associam itens a modificadores de atributos e precisam responder tanto a alterações locais quanto a consultas sobre todo o catálogo. No problema da forja de Blibblon Trombaferro, cada item modifica poucos atributos. Uma matriz densa reservaria espaço para todos os pares item–atributo, mesmo quando a maioria dos valores é zero. A matriz esparsa evita esse desperdício, mas a escolha de sua representação afeta inserções, remoções e percursos por linha e coluna.

O sistema integra três estados: o catálogo de modificadores, a fila global de encomendas e os equipamentos de cada aventureiro. Um pedido entra no fim da fila; um turno retira o primeiro pedido e empilha o item produzido no aventureiro correspondente. O desequipamento remove a peça mais recente. A *build* é o vetor que soma os modificadores dos equipamentos atuais, incluindo cada ocorrência de um item repetido.

Além dessa gestão, a consulta de recomendação ordena todo o catálogo por métricas escolhidas, agrega pontos ponderados e somente então exclui itens já equipados. Esse requisito impede calcular o ranking apenas sobre candidatos disponíveis. Os desafios principais são preservar o estado nas migrações, preservar a regra de que valores zero não ocupam células e equilibrar o custo de atualizar estruturas com o custo de consultá-las. O desempenho das estruturas depende das operações mais frequentes. Quando predominam atualizações, o custo de inserir, remover e modificar dados tem maior importância. Quando predominam consultas, a organização dos dados pode facilitar os percursos e as ordenações. Manter informações prontas para consulta reduz parte do trabalho de leitura, mas exige atualizá-las sempre que o estado do sistema muda.

A representação comprimida por linha e o índice de consultas por faixa apresentam esse compromisso. O armazenamento contíguo favorece o percurso das linhas, enquanto o índice ordenado permite localizar faixas por busca binária. Em contrapartida, inserções e remoções podem exigir deslocamentos nos vetores. A escolha entre essas estruturas depende, portanto, do tamanho do catálogo, da quantidade de modificadores e da frequência de cada operação.

## 2 Trabalhos Relacionados

Filas, pilhas, busca binária, ordenação e análise amortizada são ferramentas utilizadas para organizar estados dinâmicos. A exposição de Cormen et al. (2022) fornece a fundamentação para as operações elementares e para os limites de MergeSort e HeapSort. A discussão de ordenação por inserção de Sedgewick and Wayne (2011) destaca a influência de inversões na entrada: poucos elementos fora de posição podem reduzir o trabalho. Essa propriedade motiva comparar consultas que reiniciam a ordem por identificador com consultas que reutilizam a ordenação anterior.

Para o catálogo, Saad (2003) descreve o armazenamento comprimido por linha, que separa valores, índices de coluna e limites das linhas. Embora essa representação seja comum em computação numérica, aqui também é necessário inserir e remover células durante a execução. A discussão de hierarquia de memória de Drepper (2007) fundamenta a análise qualitativa de localidade. Essas fontes orientam a interpretação; as estruturas, algoritmos e medições apresentados a seguir correspondem à implementação específica da forja.

# 3 Métodos

## Organização e tipos abstratos de dados

O projeto usa C++11, com declarações em cabeçalhos e implementações em unidades de compilação separadas. O Makefile gera objetos em `obj` e o executável `bin/tp1.out`. Os TADs Atributo e Item armazenam identificador e nome; Aventureiro acrescenta a pilha de equipamentos. Identificadores são contadores independentes, iniciados em zero. Nomes repetidos são permitidos, pois a identidade depende do número atribuído, não do texto.

Os cadastros usam vetores dinâmicos de ponteiros, ampliados por duplicação. Pilha, Fila, Catálogo e índice de faixas são implementações próprias, sem contêineres prontos ou ponteiros inteligentes. Os nomes usam um TAD de texto com armazenamento dinâmico. A camada de comandos coordena as operações e a impressão; o Catálogo oferece consulta, atribuição, remoção, percursos e produto interno por uma interface independente do modo ativo.

Essa separação atribui uma responsabilidade a cada componente. O cadastro informa quem é o item ou aventureiro; a matriz informa o que cada item modifica; a fila informa qual pedido será atendido; a pilha informa qual equipamento será retirado primeiro. O percurso é um auxiliar que entrega uma célula por vez, sem exigir que a operação que o utiliza conheça os ponteiros de L ou os índices de T/C. Assim, a soma da build pode usar a mesma lógica nos três modos.

O programa lê comandos até o fim da entrada. Cadastros e modificadores podem ser alterados entre consultas, respeitando a existência prévia das entidades referenciadas. Uma encomenda registra os identificadores do aventureiro e do item, sem calcular poder, equilíbrio ou sinergia. Essas métricas são calculadas quando a consulta Q as solicita. Portanto, o trabalho da forja e o trabalho de recomendação têm finalidades distintas, embora ambos consultem o mesmo estado do sistema.

## Catálogo e migração

Uma célula existe exatamente quando seu valor é diferente de zero. A atribuição substitui um valor existente, insere uma célula ausente ou remove a célula quando recebe zero. Modificadores negativos são preservados.

No modo **L**, cada célula contém item, atributo, valor e dois ponteiros: próximo na linha e próximo na coluna. Vetores de cabeças permitem iniciar os percursos, mantidos em ordem crescente dos respectivos identificadores. A remoção corrige ambos os encadeamentos antes de liberar o nó. As cabeças crescem até o identificador necessário, sem duplicação de capacidade; essa decisão tem consequências para o custo de construção discutido adiante.

No modo **T**, um vetor contíguo de tripletas é ordenado primeiro por item e depois por atributo. Busca binária encontra a célula ou sua posição de inserção. A inserção desloca o sufixo para abrir espaço; a remoção fecha a lacuna. Uma linha ocupa uma sequência contígua, mas localizar seu início requer busca. Uma consulta por coluna examina as tripletas e seleciona o atributo solicitado.

O modo extra **C** usa *compressed sparse row* (CSR): dois vetores contíguos armazenam valores e identificadores de atributo, e um terceiro contém o início de cada linha, incluindo o limite final. Os limites de uma linha são obtidos diretamente; seus atributos permanecem ordenados. Inserções e remoções deslocam os vetores e ajustam os limites das linhas seguintes.

| Vetor | Conteúdo, a partir da posição 0 |
|---|---|
| Valores | $[5, -3, 7]$ |
| Atributos | $[0, 2, 1]$ |
| Inícios | $[0, 2, 2, 3]$ |

Tabela 1: Exemplo ilustrativo do modo C, com três itens e três células. O último início indica o fim dos dados.

Como exemplo, suponha que o item 0 modifique os atributos 0 e 2 com valores 5 e $-3$, o item 1 não modifique nenhum atributo e o item 2 modifique o atributo 1 com valor 7. A Tabela 1 mostra como C guarda as três células. Os vetores de valores e atributos usam a mesma posição para representar uma célula. O vetor de inícios informa qual trecho pertence a cada item; índices iguais indicam uma linha vazia.

Nesse exemplo, o item 0 ocupa as posições de 0 até antes de 2; o item 1 ocupa o intervalo vazio de 2 até antes de 2; e o item 2 ocupa a posição 2. Não existe um vetor de valores para cada item: todos compartilham os mesmos vetores, e os limites separam suas linhas. Esse acesso direto favorece o percurso, mas inserir uma célula no começo exige deslocar dados posteriores e corrigir os limites afetados. Substituir apenas um valor já existente dispensa esses deslocamentos.

A operação de migração constrói uma representação temporária, copia todas as células não nulas e somente depois libera a anterior. O índice de faixas é preservado, pois não depende do modo. A construção de C usa anexação ordenada e soma dos tamanhos de linha. Os destinos L e T reutilizam suas rotinas de inserção. Assim, preservar o estado lógico não significa que todas as direções tenham o mesmo custo.

Uma migração correta altera apenas a forma de guardar os dados. Para qualquer par item–atributo, a consulta deve devolver o mesmo valor antes e depois da troca, e células ausentes devem continuar equivalendo a zero. A fila e as pilhas permanecem válidas porque se referem a identificadores, não a endereços de células da matriz. Essa propriedade permite trocar o modo mesmo com encomendas pendentes e equipamentos em uso.

## Encomendas e vetor da build

A fila é simplesmente encadeada, com ponteiros para frente e final e um contador de pedidos. Isso permite inserir e retirar sem percorrer a fila. A pilha também é encadeada, com um único topo. A escolha evita deslocamentos e redimensionamentos nas operações de equipamentos, ao custo de ponteiros e de uma alocação por nó. Não se impõe capacidade fixa à fila nem à build.

O programa final adota **recomputação sob demanda**: equipar armazena o identificador na pilha e desequipar o remove. Em `LH` ou `Q`, um vetor de totais é zerado e recebe os

valores das linhas de todos os itens equipados. Essa escolha mantém a build coerente após qualquer `M`, inclusive quando um item equipado é modificado ou zerado. A pilha guarda ocorrências; não guarda uma cópia congelada dos modificadores no momento da fabricação.

Por exemplo, duas ocorrências de um item que concede força $+5$ produzem força total 10. Se uma peça diferente acrescenta defesa $+2$, a build passa a ter força 10 e defesa 2. Desequipar retira a peça mais recente, e a consulta seguinte recalcula os totais com as ocorrências restantes. Se o modificador de força daquele item for alterado para 3, suas duas ocorrências passam a contribuir com 6. Esse exemplo distingue o catálogo, que descreve cada tipo de item, da build, que depende do que um aventureiro possui naquele momento.

A política da fila é *first in, first out* (FIFO): o primeiro pedido recebido é o primeiro atendido. A da pilha é *last in, first out* (LIFO): a última peça equipada é a primeira removida. Um comando Z pode atender pedidos de aventureiros diferentes, mas cada peça vai para a pilha de seu solicitante. Se a fila acabar, os turnos restantes não alteram o estado. Se uma pilha estiver vazia, D apenas informa essa condição. Essas regras dispensam procurar um equipamento no meio da estrutura.

Na bancada, uma alternativa **imediata** soma a linha ao equipar e a subtrai ao desequipar. Uma alteração `M` percorre as pilhas dos aventureiros e aplica a diferença a cada ocorrência do item. Portanto, antecipar o cálculo também aumenta o custo de atualizar o catálogo. Ambas as variantes experimentais reutilizam vetores auxiliares, para evitar que somente uma delas seja penalizada por alocações auxiliares.

## Recomendação e operações numéricas

Sejam $N$ o número de itens, $T$ o número de atributos, $v_i(a)$ o modificador do item $i$ e $b_h(a)$ o total da build do aventureiro $h$. A implementação calcula:

$$P_i = \sum_a v_i(a), \quad \mu_i = P_i/T,$$

$$E_i = \frac{1}{1 + \sqrt{\frac{1}{T}\sum_a (v_i(a) - \mu_i)^2}}, \tag{1}$$

$$S_{ih} = \frac{\sum_a v_i(a) b_h(a)}{\|v_i\|_2 \, \|b_h\|_2}. \tag{2}$$

No equilíbrio, os atributos ausentes contribuem com $\mu_i^2$ cada um, sem criar um vetor com todos os seus zeros. Se $T = 0$, o resultado é 1. A norma é a raiz da soma dos quadrados dos componentes de um vetor. Na sinergia, norma nula resulta em zero; o resultado é limitado a $[-1, 1]$ para conter desvios numéricos. Totais da build usam `long long`, e as métricas usam `double`.

Poder mede o saldo dos modificadores, incluindo os negativos. Equilíbrio mede a distribuição desses valores entre todos os atributos cadastrados: atributos sem célula também contam, com valor zero. Sinergia compara a direção do vetor de um item com a direção da build. Um resultado positivo indica alinhamento; um negativo indica oposição; zero pode significar ausência de contribuição comum ou um dos vetores nulo. Como a build muda, a sinergia de um item pode mudar sem que sua própria linha seja alterada.

Cada valor de métrica é arredondado para seis casas antes da comparação. MergeSort ordena em ordem decrescente, com identificador crescente no desempate. Se $r_j(i)$ é a posição a partir de zero na métrica $j$ e $w_j$ é seu peso, a pontuação é

$$\mathrm{pontuacao}(i) = \sum_{j=1}^{m} w_j \big(N - r_j(i)\big). \tag{3}$$

Os $N$ itens participam das $m$ ordenações. Depois, um vetor booleano marca os equipados, os candidatos restantes são ordenados por pontuação e identificador, e até $n$ resultados são impressos com duas casas decimais. O programa não mantém ordens entre consultas. A bancada acrescenta essa alternativa, além de Inserção e HeapSort, sem alterar o programa submetido.

Os pontos dependem da posição, e não diretamente do valor bruto da métrica. Em um catálogo de quatro itens, as posições recebem 4, 3, 2 e 1 pontos. Um item em segundo lugar em uma métrica de peso 2 recebe 6 pontos dessa métrica. As contribuições são somadas para chegar à pontuação final. Um item já equipado ainda participa dessas ordenações e ocupa sua posição; ele só é excluído ao montar a resposta. Retirá-lo antes mudaria os pontos dos demais e produziria outra regra de recomendação.

O arredondamento para seis casas torna o desempate reproduzível diante de pequenas diferenças de ponto flutuante. Ele ocorre ao comparar métricas; a saída final usa duas casas. Uma build com totais que se cancelam é tratada como vetor nulo, mesmo que a pilha contenha peças. Se não houver candidatos após a filtragem, Q não imprime linhas. Já LI, LA e LH indicam VAZIO quando não têm valores a listar.

O Catálogo também expõe o produto interno entre duas linhas esparsas por intercalação dos atributos ordenados. Na recomendação final, a sinergia utiliza diretamente a linha esparsa e a build densa; a build é formada por somas de linhas. Logo, o ensaio de produto entre duas linhas compara um núcleo numérico relacionado, não substitui automaticamente o cálculo completo de `Q`.

## Consulta por faixa e robustez

Para cada atributo, o índice de `LF` guarda um vetor de pares não nulos ordenado por valor e identificador. Duas buscas binárias localizam o primeiro valor maior ou igual ao mínimo e o primeiro estritamente maior que o máximo. Os itens ausentes são representados por intervalos disjuntos de identificadores. Quando zero pertence à faixa, esses intervalos são enumerados e acrescentados aos resultados. HeapSort próprio ordena a resposta por identificador.

Uma atualização remove o par antigo e insere o novo no vetor. Ao passar de zero para não zero, retira o identificador dos intervalos de zeros, podendo dividir um intervalo. Na transição inversa, devolve o identificador e une intervalos adjacentes quando necessário. Novos itens e atributos também são registrados no índice. Essa organização inclui tanto células removidas quanto itens que nunca tiveram modificador naquele atributo.

Considere, por exemplo, um atributo em que o item 0 vale 5, o item 1 não tem célula e o item 2 vale $-2$. A consulta LF na faixa $[-2, 0]$ deve retornar os itens 1 e 2, nessa ordem de identificador. Se M atribuir zero ao item 0, sua célula desaparece da matriz, mas o item passa a pertencer ao conjunto de zeros do índice. A mesma consulta deve então incluir os três itens. O exemplo mostra por que consultar somente as células armazenadas não basta para responder LF corretamente.

Ordenar o índice por valor permite encontrar rapidamente os limites numéricos, mas a saída exige ordem por identificador. Por isso, a consulta tem duas etapas: selecionar o trecho de valores e ordenar os resultados obtidos. Essas ordenações usam critérios diferentes. O índice não elimina o trabalho proporcional ao tamanho da resposta; sua finalidade é evitar examinar todo o catálogo quando a faixa contém poucos resultados.

Embora a entrada seja válida por contrato, a implementação verifica posições negativas e uso de rotinas incompatíveis com o modo. Cópias de classes proprietárias de ponteiros são desabilitadas em estruturas centrais, e destrutores liberam nós e vetores. Pilha e fila vazias são tratadas sem acesso inválido. Essas medidas não constituem garantia geral contra falhas de alocação ou estouro: os limites dos tipos inteiros continuam sendo um pressuposto operacional.

### Entrada, saída e uso da memória

A leitura usa o formato de cada comando para obter seus parâmetros e acionar a operação correspondente. O programa não precisa recuperar entradas malformadas, pois o enunciado garante sua validade. Isso permite concentrar o tratamento de casos especiais no estado dos dados: catálogo sem células, fila vazia, item repetido na pilha, ausência de candidatos e totais que se anulam. Nomes podem coincidir, mas uma alteração sempre usa identificadores, evitando confundir entidades diferentes com o mesmo texto.

A saída é montada segundo a ordem exigida pelo comando. LI percorre atributos por identificador; LA percorre itens por identificador; LH percorre os totais e omite os zeros. Em Z, a primeira linha informa quantas encomendas foram atendidas, e a segunda, quando necessária, mantém a ordem em que saíram da fila. Essa distinção entre a ordem dos dados internos e a ordem exigida na saída também explica a ordenação final de LF e Q.

O gerenciamento manual de memória acompanha a responsabilidade de cada TAD. Retirar uma encomenda libera o nó da fila; desequipar libera o nó da pilha, mas não remove o item do cadastro. Um vetor temporário de resposta pertence à operação que o recebe e deve ser liberado depois de usado. Essa separação evita confundir a existência de um item no catálogo com suas ocorrências nos equipamentos. A implementação não utiliza um objeto de item completo em cada nó da pilha: guarda seu identificador.

## 4 Análise de Complexidade

Sejam $N$ itens, $T$ atributos, $S$ células não nulas, $s_i$ células na linha $i$, $c_a$ na coluna $a$, $b_h$ peças do aventureiro e $S_h$ a soma dos tamanhos de suas linhas, contando repetições. Considere $\ell(x) = 1 + \log_2(x + 1)$. Os custos da Tabela 2 excluem impressão, crescimento de cabeças e manutenção do índice de faixas, explicitados separadamente.

A fila e a pilha realizam suas operações elementares em $O(1)$, no modelo usual que abstrai o custo do alocador. Assim, `F` e `D` custam $O(1)$ e `Z` custa $O(1 + \min(k, q))$, sendo $q$ o tamanho da fila. O cadastro em vetor de ponteiros tem custo amortizado constante, além da cópia do nome; porém, cadastrar um item também atualiza os intervalos de todas as $T$ colunas do índice, com custo usual $O(T)$, podendo haver realocações.

Na Tabela 2, a diferença entre substituir e inserir é essencial. Em T/C, encontrar uma célula por busca binária não torna sua inserção logarítmica: abrir espaço ainda pode mover quase todo o vetor. Em L, conectar ou desconectar um nó exige poucas alterações de ponteiros, mas antes é necessário localizar seus vizinhos na linha e na coluna. A tabela inclui essa procura. Assim, dizer apenas que a lista insere em tempo constante ignoraria uma parte do trabalho desta implementação.

O termo $T$ no cálculo da build corresponde à criação e à inicialização de um total por atributo, inclusive quando a maioria ficará em zero. O termo $b_h$ conta as ocorrências da pilha, e $S_h$ conta os modificadores somados. Em T, cada ocorrência ainda exige localizar o início da linha. Itens repetidos não podem ser simplesmente ignorados: sua contribuição precisa ser multiplicada pelo número de ocorrências, aqui processadas individualmente.

Se $z_a$ é o número de intervalos de zeros, manter o índice de um atributo custa até $O(c_a + z_a + 1)$ por alteração, devido aos deslocamentos de vetores. Como $z_a \le c_a + 1$, o custo é $O(c_a + 1)$, limitado por $O(N)$. Para $K$ resultados, `LF` custa $O(\ell(c_a) + K \log(K + 1))$ e usa $O(K)$ espaço temporário. A enumeração dos zeros é proporcional aos resultados correspondentes, não uma varredura de todos os itens seguida de filtragem. Em particular, uma faixa que retorna quase todo o catálogo necessariamente exige produzir quase $N$ pares.

Com espaço já reservado, o produto intercalado custa $O(1 + s_i + s_j)$ em L/C e acrescenta $O(\ell(S))$ em T. A alternativa densa custa $O(T + s_i + s_j)$, mais a localização das linhas, e usa $O(T)$ memória auxiliar. A limpeza do vetor denso faz parte desse custo.

O poder e o equilíbrio de todo o catálogo custam $O(N + S)$ em L/C; em T, as buscas por início de linha acrescentam $O(N\ell(S))$. Na implementação da sinergia, o vetor da build é percorrido novamente para calcular sua norma a cada item, acrescentando $O(NT)$. Se $B_h$ denota o custo de recomputar a build e $\delta = 1$ quando SINERGIA é solicitada, um limite para `Q` em L/C é

$$O\big(B_h + m(N + S) + \delta NT + (m + 1)N\ell(N)\big). \qquad (4)$$

Em T soma-se $O(mN\ell(S))$. O termo de ordenação inclui as métricas e a ordenação final dos candidatos. Pedir poucos resultados reduz a saída, mas não elimina as ordenações completas. O espaço auxiliar de `Q` é $O(N + T)$.

As estratégias imediata e sob demanda distribuem esse custo de formas diferentes. Na versão imediata da bancada, equipar ou retirar uma peça percorre a linha correspondente;

| Operação | L: lista cruzada | T: tripletas | C: comprimida por linha |
|---|---|---|---|
| Consultar célula | $O(1 + s_i)$ | $O(\ell(S))$ | $O(\ell(s_i))$ |
| Substituir valor não nulo | $O(1 + s_i)$ | $O(\ell(S))$ | $O(\ell(s_i))$ |
| Inserir/remover célula | $O(1 + s_i + c_a)$ | $O(1 + S)$ | $O(1 + S + N)$ |
| Percorrer linha (LI) | $O(1 + s_i)$ | $O(\ell(S) + s_i)$ | $O(1 + s_i)$ |
| Percorrer coluna (LA) | $O(1 + c_a)$ | $O(1 + S)$ | $O(\sum_i \ell(s_i))$ |
| Recomputar build (LH) | $O(T + b_h + S_h)$ | $O(T + b_h\ell(S) + S_h)$ | $O(T + b_h + S_h)$ |

Tabela 2: Custos das representações implementadas. Inserção e remoção indicam pior caso; uma remoção inexistente pode terminar antes. A manutenção de LF acrescenta custo a M em todos os modos.

consultar o vetor já atualizado percorre seus $T$ totais. Uma alteração M também precisa procurar ocorrências do item nas pilhas. Se $B$ é o total de peças de todos os aventureiros e $H$ seu número, essa etapa custa $O(H + B)$, além da atualização do catálogo. Portanto, reduzir LH pode tornar mais caras operações que antes apenas alteravam uma pilha ou uma célula.

Para a ordenação, MergeSort e HeapSort têm limite de pior caso $O(N \log N)$; Inserção pode atingir $O(N^2)$. Entretanto, uma sequência quase ordenada tem poucas inversões, isto é, poucos pares fora da ordem desejada, e pode ser reorganizada por Inserção com muito menos deslocamentos. Esse benefício só aparece se a ordem aproveitada tiver relação com a nova consulta. A ordenação dos candidatos, reconstruídos por identificador, não necessariamente possui essa propriedade.

As migrações para C custam $O(N + S)$ a partir de L e $O(N\ell(S) + S)$ a partir de T. Para T, a inserção de células já ordenadas evita deslocar sufixos, mas ainda realiza buscas, resultando em $O(N + S\ell(S))$ a partir de L/C. Reconstruir L por inserções sucessivas pode custar

$$O\left(N^2 + T^2 + \sum_i s_i^2 + \sum_a c_a^2\right), \tag{5}$$

além do percurso da origem. O limite inclui a expansão exata das cabeças e a procura do final das listas. Portanto, não se atribui custo linear a uma migração cuja implementação não usa ponteiros de cauda nem reserva prévia de todas as cabeças.

A representação L usa $O(N + T + S)$ espaço; T usa $O(S)$ e C, $O(N + S)$. O índice de faixas usa $O(S + T)$, incluindo intervalos. Esses limites descrevem o estado sem capacidades excedentes; os vetores retêm capacidade após remoções, de modo que a memória alocada também depende do maior tamanho anterior. Somando cadastros, fila e pilhas, o sistema usa espaço linear no catálogo, nas entidades e nas ocorrências de equipamentos, além dos nomes e dos buffers temporários.

Para interpretar os limites de espaço, é útil separar células existentes de combinações possíveis. Uma matriz densa usa uma posição para cada um dos $NT$ pares; as representações esparsas guardam os $S$ valores não nulos e estruturas de acesso. Na base de 256 itens e 128 atributos, há 32.768 pares possíveis, mas uma densidade de aproximadamente 5% ocupa apenas uma pequena fração deles. A economia não depende de todos os valores serem positivos: tanto 5 quanto $-3$ exigem uma célula, enquanto zero não exige.

O índice de faixas não precisa criar um par explícito para cada zero. Em uma coluna com poucos valores não nulos, longas sequências de itens ausentes podem ser descritas por poucos intervalos. Cada valor presente separa, no máximo, dois trechos de zeros, o que fundamenta o limite $z_a \leq c_a + 1$. Entretanto, ao pedir todos esses zeros em LF, a resposta volta a precisar de um par por item retornado. Economizar a representação interna não elimina o tamanho exigido para a saída.

# 5 Análise Experimental

## Ambiente, cargas e controle das medições

Os dados foram coletados em Intel Core i9-12900K, com 31,7 GiB de RAM, Windows 11 Pro 64 bits, versão 10.0.26200, e MinGW GCC 6.3.0. A compilação usou C++11 e `-O2`, com avisos habilitados, sem LTO. Apesar do sistema operacional de 64 bits, o executável medido tem ponteiros de quatro bytes. Tempos usam QueryPerformanceCounter. Na calibração de 100.000 intervalos vazios, a média foi 5,699 ns; 94.438 intervalos foram zero e o menor positivo observado foi 100 ns. Esses números caracterizam a coleta, não uma resolução universal; o custo do relógio não foi subtraído.

Cada configuração teve cinco repetições temporais e uma execução separada com contadores. São apresentadas medianas das médias por operação; a Figura 1 também apresenta quartis. Nos perfis, a semente do catálogo é $773 + 97r$, para repetição $r$, e a carga usa a semente seguinte. As entradas são pareadas entre modos e estratégias, cuja ordem de execução é rotacionada. Assim, a dispersão inclui variação da entrada, além do ambiente.

O cenário base contém 256 itens, 128 atributos, densidade solicitada de 5%, oito aventureiros, 16 peças iniciais por build e 32 pedidos na fila; em Q, $n = 5$ e $m = 3$. Os valores iniciais são não nulos entre $-9$ e 9. Cada perfil contém 1.200 comandos. No perfil de atualizações, as proporções de M/F/Z/D são 30/20/20/20%, e de LI/LA/LH/Q, 2/2/3/3%. No perfil de consultas, são 4/2/2/2% e 20/20/25/25%, respectivamente. As proporções são exatas em blocos embaralhados de 100 comandos; Z processa um turno. Em M são gerados valores entre $-9$ e 9, com probabilidade adicional de zerar a célula.

As configurações da Tabela 3 não formam um experimento fatorial completo. As densidades iniciais efetivas da

| Parâmetro | Base | Valores adicionais |
|---|---|---|
| Itens $N$ | 256 | 64; 1.024 |
| Atributos $T$ | 128 | 32; 512 |
| Densidade | 5% | 0,5%; 25% |
| Peças iniciais/build | 16 | 1; 64 |
| Pedidos iniciais | 32 | 0; 256 |
| Resultados $n$ | 5 | 1; 50 |
| Métricas $m$ | 3 | 1; 2 |

Tabela 3: Variações dos perfis: um fator por vez, totalizando 15 configurações. Cada uma é executada nos dois perfis, três modos e duas estratégias de build.

primeira semente foram 4,898% na base, 0,531% no cenário mais esparso e 24,762% no mais denso. Na base, as medianas da build média observada foram 16,003 e 16,009 peças, e da fila média, 32,213 e 32,017 pedidos, nos perfis de atualizações e consultas. A densidade final mediana foi 5,722% e 5,032%, respectivamente.

A bancada compila separadamente os módulos reais de TP. Um motor experimental coordena os mesmos TADs e funções de métricas, mas elimina a entrada/saída textual e reutiliza buffers nas duas estratégias. Em M, ele também consulta o valor anterior para permitir a atualização imediata. Portanto, os tempos medem esse caminho experimental, não o tempo completo do executável lendo e imprimindo comandos. Geradores e preparação inicial ficam fora dos intervalos; alocações inerentes às operações, como nós e respostas de LF, permanecem incluídas. Os totais somam intervalos por operação, sem a preparação inicial. Assinaturas dos resultados e verificações de builds detectam divergências entre variantes.

Uma média por operação é obtida dividindo o tempo acumulado pelo número de execuções daquele comando. Para resumir as cinco repetições, usa-se a mediana dessas médias, que sofre menos influência de uma repetição especialmente lenta. P25 e P75 são os quartis: delimitam a metade central dos valores observados. As barras dos gráficos mostram essa dispersão e não representam intervalos de confiança. Como também há mudança de semente, não separam completamente a variação do ambiente da variação da carga.

Os contadores medem trabalho lógico, como células visitadas e comparações, e são habilitados em uma compilação separada. Essa separação evita somar o custo dos incrementos aos tempos principais. Os TADs armazenam as contagens e fornecem operações para consultá-las e reiniciá-las. A memória é estimada a partir dos objetos e das capacidades de seus vetores. Portanto, as três medidas respondem a perguntas complementares: quanto tempo a operação levou, quanto trabalho foi contado e quanto espaço as estruturas reservaram.

## Perfis, memória e migração

A Tabela 4 mostra vantagem de L nas operações de catálogo e consulta da base. No perfil de consultas, Q em T custou 28,5% a mais que em L; os intervalos interquartis foram 102,103–103,146 $\mu$s em L, 128,243–137,282 em T e

| Perfil | Operação | L | T | C |
|---|---|---|---|---|
| Atualiz. | M | 0,217 | 0,521 | 0,428 |
| | F | 0,060 | 0,064 | 0,062 |
| | Z | 0,077 | 0,076 | 0,077 |
| | D | 0,050 | 0,048 | 0,047 |
| | LI | 0,083 | 0,142 | 0,104 |
| | LA | 0,121 | 0,950 | 3,425 |
| | LH | 0,744 | 1,447 | 0,858 |
| | Q | 112,811 | 150,756 | 123,358 |
| Consultas | M | 0,258 | 0,571 | 0,438 |
| | F | 0,121 | 0,129 | 0,138 |
| | Z | 0,138 | 0,171 | 0,154 |
| | D | 0,100 | 0,096 | 0,113 |
| | LI | 0,074 | 0,114 | 0,088 |
| | LA | 0,098 | 0,870 | 3,237 |
| | LH | 0,677 | 1,240 | 0,791 |
| | Q | 103,001 | 132,398 | 110,683 |

Tabela 4: Medianas em $\mu$s por operação na configuração base, com build sob demanda. M inclui manutenção do índice de LF.

109,455–116,183 em C. A coluna tem efeito especialmente claro: LA percorre apenas os nós pertinentes em L, examina todas as tripletas em T e realiza buscas nas linhas em C.

Na coleta instrumentada, LA examinou em média 12,65 células por consulta em L e 1.621,26 em T. Em C, retornou 12,65 células, mas efetuou 725,58 comparações de busca. Q registrou 6.586,60 células em L/C e 7.622,62 em T, com 11.172,96 comparações de busca em T. Os contadores não registram exatamente o mesmo tipo de visita em todas as representações: em T, a verificação que encerra uma linha também pode contar uma tripleta; em C, buscas contam comparações e percursos contam células devolvidas. Esses valores não são medidas de acessos físicos nem de falhas de cache.

Os totais medianos das operações foram 4,212/5,737/4,742 ms em L/T/C no perfil de atualizações e 31,168/40,363/34,263 ms no de consultas. Mesmo no primeiro perfil, apenas 36 chamadas de Q em L representam aproximadamente 4,06 ms; por isso a frequência de atualizações não implica que elas dominem o tempo total.

Essa diferença também orienta a interpretação das médias. Melhorar alguns nanossegundos em F ou D tem efeito limitado no total quando uma chamada de Q custa centenas de vezes mais. Por outro lado, uma aplicação com quase nenhuma recomendação pode ser mais sensível às alterações e aos percursos simples. Por isso foram mantidas as duas formas de apresentar os resultados: o tempo por comando ajuda a explicar a estrutura; o tempo agregado mostra o efeito da combinação de comandos daquela carga.

A Tabela 5 evidencia o custo em memória: C reduz em 23,4% a estimativa total do catálogo em relação a L. O índice compartilhado de LF custa mais que a representação principal em cada modo nessa carga. A estimativa inclui capacidades reservadas, mas exclui metadados do alocador, fragmentação, textos dinâmicos dos nomes e picos tran-

| Modo | Catálogo | Representação | Índice LF |
|---|---|---|---|
| L | 74.948 | 34.516 | 39.872 |
| T | 64.568 | 24.576 | 39.872 |
| C | 57.404 | 17.412 | 39.872 |

Tabela 5: Memória estimada em bytes ao final da carga base de consultas, build sob demanda; medianas. Catálogo inclui seu objeto; as outras colunas incluem apenas alocações dinâmicas.

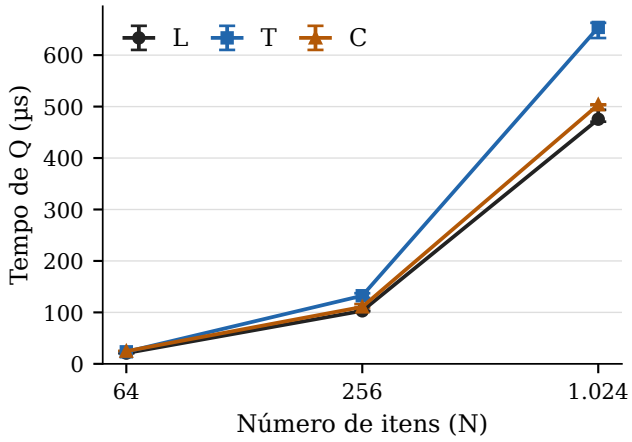



Figura 1: Q no perfil de consultas, build sob demanda, variando $N$ e mantendo os demais parâmetros da base. Pontos: medianas; barras: P25–P75 das cinco médias por repetição.

sitórios. Não mede toda a memória ocupada pelo processo no sistema operacional. O pico estável estimado do motor nessa configuração foi 109.124, 98.760 e 91.596 bytes em L/T/C.

Nas migrações da base, as medianas foram 95,2 $\mu$s para L→T, 64,1 para L→C, 262,4 para T→L, 28,4 para T→C, 219,7 para C→L e 64,2 para C→T. Cada direção partiu de uma matriz nova com dados pareados, incluindo a liberação da origem no intervalo. Reconstruir listas por inserções repetidas custou mais que preencher C em ordem. A troca de modo, portanto, só se justifica quando operações futuras compensam esse custo.

A Figura 1 mostra o crescimento com o catálogo: para 64 e 1.024 itens, Q em L passou de 20,856 para 475,436 $\mu$s. Com 32 e 512 atributos, os valores foram 71,117 e 210,092 $\mu$s; elevar a densidade solicitada de 0,5% para 25% elevou Q de 76,121 para 268,947 $\mu$s. São efeitos compatíveis com mais células, mais ordenação e o termo $NT$ da sinergia.

Os cenários com $m = 1, 2, 3$ usam, respectivamente, PODER; PODER e EQUILÍBRIO; e as três métricas, com pesos 1, 2 e 3. Assim, variar $m$ também muda o trabalho numérico. Alterar $n$ de 1 para 50 não remove as ordenações completas. A fila afeta F/Z, mas seu comprimento não é percorrido por Q. Os blocos de build, fila e $n$ apresentam patamares temporais maiores que a base, inclusive

| Modo | Operação | Demanda | Imediata |
|---|---|---|---|
| L | M | 0,258 | 0,467 |
| | Z | 0,138 | 0,179 |
| | D | 0,100 | 0,121 |
| | LH | 0,677 | 0,195 |
| | Q | 103,001 | 102,613 |
| T | M | 0,571 | 0,752 |
| | Z | 0,171 | 0,229 |
| | D | 0,096 | 0,171 |
| | LH | 1,240 | 0,190 |
| | Q | 132,398 | 127,937 |
| C | M | 0,438 | 0,644 |
| | Z | 0,154 | 0,221 |
| | D | 0,113 | 0,163 |
| | LH | 0,791 | 0,196 |
| | Q | 110,683 | 111,480 |

Tabela 6: Estratégias de build na base de consultas; medianas em $\mu$s. O vetor imediato reduz LH, mas aumenta M, Z e D.

quando não se espera efeito direto em Q; como a ordem das configurações não foi aleatorizada, essas diferenças não permitem atribuição causal isolada a cada parâmetro.

### Manutenção imediata ou sob demanda

A Tabela 6 confirma a transferência de trabalho. Em L, LH caiu de 0,677 para 0,195 $\mu$s, enquanto M aumentou de 0,258 para 0,467 $\mu$s. No perfil de atualizações em L, Z passou de 0,077 para 0,111 $\mu$s e D de 0,050 para 0,073 $\mu$s; LH caiu de 0,744 para 0,200 $\mu$s. Q mudou pouco porque seu custo é dominado por métricas e ordenações. A pequena piora de Q em C não demonstra que antecipar a build seja intrinsecamente pior; exige considerar a dispersão e o restante da consulta.

No ensaio isolado de 50.000 operações, a build inicia com 16 peças e alterna equipar/desequipar entre consultas do vetor. O catálogo fica fixo, sem M, fila ou Q. Foram amostradas frequências de consulta de 0, 1, 2, 5, 10, 25, 50, 75 e 90%. Em todos os modos, a imediata ainda perdeu em 5% e venceu em 10%. Em L, os totais demanda/imediata foram 3,154/3,551 ms em 5% e 4,749/3,731 ms em 10%. O cruzamento observado está entre esses pontos, não em um limiar universal. As medições de F/Z/D e as variações de fila/build avaliam a implementação encadeada; não houve comparação direta com uma implementação por vetor.

A Figura 2 apresenta a região que contém essa mudança de vantagem. Sem consultas, a estratégia sob demanda evita o trabalho de somar e subtrair modificadores. Conforme as consultas se tornam mais frequentes, reconstruir o vetor repetidamente passa a custar mais. Na estratégia imediata, uma parte desse trabalho já foi realizada durante as trocas de equipamento. As linhas apenas conectam as frequências medidas; não permitem localizar com precisão o cruzamento entre 5 e 10%. O gráfico ilustra o modo L, enquanto a verificação desse intervalo também foi realizada nos modos T e C.

Esse resultado ajuda a interpretar a escolha final. Se o aventureiro troca equipamentos muitas vezes antes de con-

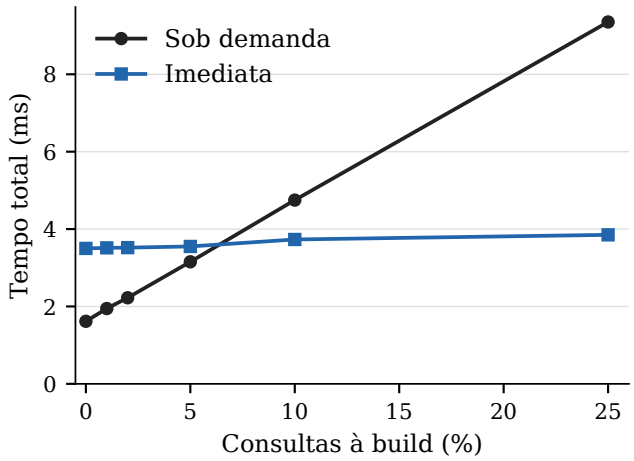



Figura 2: Tempo mediano de 50.000 operações no modo L, com catálogo fixo. Recorte de 0 a 25% de consultas à build; os demais comandos alternam equipar e desequipar.

| Algoritmo | Reusa | Q | Métricas | Compar. |
|---|---|---|---|---|
| Merge | não | 385,89 | 119,56 | 11.898 |
| Inserção | não | 1.118,92 | 690,51 | 193.724 |
| Heap | não | 430,88 | 156,43 | 22.928 |
| Merge | sim | 318,71 | 52,54 | 7.011 |
| Inserção | sim | 448,69 | 5,26 | 1.716 |
| Heap | sim | 400,13 | 126,68 | 23.883 |

Tabela 7: Ordenadores em L, $N = 512$, $T = 128$, densidade 5%, três métricas e uma alteração M entre consultas. Q e ordenação das métricas em $\mu$s; comparações são médias por Q, arredondadas, da coleta instrumentada.

sultar sua build, recomputar apenas na consulta evita manter totais que não serão usados. Se consulta a mesma build várias vezes, antecipar as somas pode compensar. Porém, não basta contar Q como se fosse LH: Q contém outras etapas muito mais caras. Da mesma forma, o limiar do ensaio com catálogo fixo não prevê sozinho o resultado de uma carga com muitas alterações M.

## Ordenação e reaproveitamento de ordens

O ensaio usa $N \in \{128, 512\}$, zero, uma ou 16 alterações M entre consultas, uma consulta inicial de aquecimento e 40 consultas medidas. MergeSort oferece $O(N \log N)$ previsível com buffer auxiliar; HeapSort tem o mesmo limite e dispensa um vetor auxiliar próprio; Inserção pode aproveitar poucas inversões, embora seu pior caso seja quadrático. Todos usam o mesmo desempate.

Na Tabela 7, reutilizar a ordem reduziu as comparações das métricas por Inserção de 193.724 para cerca de 1.716. Entretanto, o melhor tempo de Q permaneceu com MergeSort. A bancada também aplica o ordenador escolhido à lista final de candidatos, reconstruída por identificador e sem reaproveitamento. Por isso, a redução para 5,26 $\mu$s nas ordenações de métricas não torna toda a consulta mais

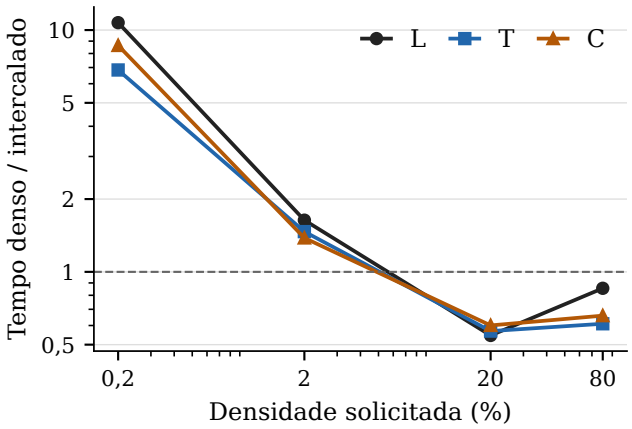



Figura 3: Razão entre medianas do tempo denso e intercalado, com $T = 4.096$. Acima de 1, a intercalação é mais rápida; abaixo, o vetor denso é mais rápido. Densidades solicitadas em escala logarítmica.

rápida. Um algoritmo híbrido, com Inserção nas métricas quase ordenadas e MergeSort nos candidatos, é uma possibilidade futura, não uma variante medida. Além disso, uma alteração de item equipado pode mudar a sinergia de muitos itens: poucas operações M não garantem poucas inversões.

## Produto interno e representação comprimida

O produto interno foi medido em 256 pares predefinidos de um catálogo de 64 itens, com $T \in \{64, 512, 4096\}$ e densidades solicitadas de 0,2%, 2%, 20% e 80%. O buffer denso é alocado antes do lote, mas zerado dentro do cálculo. Ambos os produtos são compilados em unidades separadas, sem LTO, e os resultados são consumidos e comparados.

Na Figura 3, a intercalação evita a limpeza de milhares de posições quando há poucos modificadores. Em L, com 0,2% de densidade, os tempos denso/intercalado foram 1,316/0,123 $\mu$s; com 20%, foram 6,485/11,881 $\mu$s. Na maior densidade, T apresentou 24,684/40,464 $\mu$s. Nesse caso, o produto denso em T foi mais rápido que em L, que consumiu 34,688 $\mu$s: a vantagem de L nos perfis não se estende a todos os núcleos e densidades. O buffer denso ocupa 32.768 bytes para $T = 4.096$, enquanto a intercalação não aloca esse vetor. O cruzamento observado entre 2% e 20% depende de $T$, da representação e dos acessos, não apenas da notação assintótica.

O ensaio específico dos extras usa cinco configurações: $(N, T, \rho) = (256, 128, 0{,}02)$, $(256, 128, 0{,}20)$, $(1024, 128, 0{,}02)$, $(1024, 512, 0{,}02)$ e $(1024, 512, 0{,}10)$, com cinco repetições. A semente é $20261022 + 100c + r$, para configuração $c$ e repetição $r$. A Tabela 8 mostra que C pode substituir um valor sem deslocar os demais, mas inserir ou remover no vetor global é caro. Na maior configuração, sua inserção custou 12,213 $\mu$s, contra 1,050 em L. O percurso contíguo de C não venceu L nessas medições. Já a substituição custou 0,182 $\mu$s em C e 0,282 em L: não há um vencedor único para todas as operações.

| Modo | $N$ | $T$ | Densidade | Percurso/linha | Inserção | Substituição | Remoção |
|---|---|---|---|---|---|---|---|
| L | 256 | 128 | 2,0% | 0,011 | 0,154 | 0,063 | 0,084 |
| T | 256 | 128 | 2,0% | 0,016 | 0,326 | 0,160 | 0,226 |
| C | 256 | 128 | 2,0% | 0,017 | 0,284 | 0,064 | 0,249 |
| L | 1.024 | 512 | 10,1% | 0,152 | 1,050 | 0,282 | 0,787 |
| T | 1.024 | 512 | 10,1% | 0,189 | 11,046 | 0,320 | 10,834 |
| C | 1.024 | 512 | 10,1% | 0,189 | 12,213 | 0,182 | 12,002 |

Tabela 8: Recortes do ensaio específico de C. Medianas em $\mu$s por linha percorrida ou célula alterada, incluindo manutenção de LF. Densidades efetivas arredondadas.

### Consultas por faixa e valores zero

Com $N = 4.096$, $T = 32$ e densidade solicitada de 2%, a consulta na faixa $[2, 4]$ retornou uma mediana de 12 itens. Os tempos foram 0,208, 0,206 e 0,209 $\mu$s nos modos L, T e C, respectivamente. Para a faixa $[0, 0]$, a mediana foi de 4.025 itens, com tempos de 222,759, 217,592 e 217,311 $\mu$s. Esse aumento está relacionado à quantidade de resultados que precisam ser copiados e ordenados. Na coleta separada de contadores, a consulta $[0, 0]$ realizou 12 comparações para localizar os limites e copiou 4.017 resultados. Essa quantidade corresponde à semente daquela execução, enquanto a mediana resume as cinco sementes utilizadas nas medições de tempo.

As configurações de LF também incluíram catálogos com 256 itens, densidade de 50% e consultas nas faixas $[-1, 1]$ e $[100, 200]$, com 64 consultas por lote. Para $N = 4.096$ e densidade de 2%, a faixa $[100, 200]$ não retornou itens e apresentou tempo próximo de 0,020 $\mu$s nos três modos. Os tempos semelhantes estão relacionados ao uso do mesmo índice de faixas, que funciona independentemente da representação da matriz.

O custo da consulta depende principalmente da busca pelos limites e da quantidade $K$ de itens retornados. Embora $[0, 0]$ contenha apenas um valor, essa faixa pode incluir quase todo o catálogo quando a matriz possui muitos zeros. Uma faixa mais ampla, por outro lado, pode retornar poucos itens quando seus valores são raros. A busca binária localiza os limites sem percorrer todo o catálogo, mas copiar e ordenar uma resposta extensa continua exigindo mais trabalho.

### Localidade de referência

No modo T, as tripletas ficam em posições consecutivas na memória. No modo C, os valores e os atributos também são armazenados em vetores contíguos, e os limites de cada linha são obtidos diretamente. Essa organização favorece a localidade espacial, pois dados próximos podem ser carregados juntos para o cache. No modo L, o percurso segue ponteiros entre nós que podem estar espalhados pela memória. A localidade temporal ocorre quando consultas sucessivas reutilizam dados do catálogo ou vetores auxiliares acessados recentemente (Drepper 2007). O benefício desses acessos depende também da quantidade de operações necessárias.

Na consulta Q, o cálculo do equilíbrio percorre cada linha para obter o poder e novamente para calcular os desvios. A sinergia percorre a build para cada item, enquanto T realiza buscas binárias para localizar o início das linhas. Em catálogos pequenos e esparsos, essas operações adicionais podem reduzir a vantagem do armazenamento contíguo. Na consulta LA, T examina as $S$ tripletas do catálogo, enquanto L percorre apenas os $c_a$ nós associados ao atributo solicitado. Assim, acessar menos elementos pode ser mais vantajoso do que percorrer sequencialmente um vetor maior.

Nas inserções e remoções, T e C podem deslocar vários elementos, mesmo acessando posições vizinhas na memória. Em L, depois de localizar a posição, a inserção exige ajustar os ponteiros, sem deslocar as células já armazenadas. Na maior configuração testada, a inserção levou 12,213 $\mu$s em C e 1,050 $\mu$s em L, resultado coerente com o custo dos deslocamentos. Como não foram utilizados contadores de hardware, a influência do cache é discutida a partir desses padrões de acesso, sem quantificar as falhas de cache de cada representação.

### Validação e limites da avaliação

A preparação validou TADs, produtos internos, migrações e consultas por faixa, inclusive zeros. O verificador de comandos comparou 13.971 comandos em nove execuções, variando sementes e modos iniciais, com uma referência independente. A investigação adicional da remoção de células verificou 137.781 transições entre valores $-1$, 0 e 1 em estados de sete itens, com migrações, e mais 74.793 comandos direcionados a LF, sem divergências.

As avaliações automáticas principal e extra obtiveram 100/100 nos conjuntos de testes (VPL's), complementando os testes locais.

Os tempos obtidos dependem do computador utilizado e das condições de execução. Foram realizadas cinco repetições, sem fixar o núcleo de execução ou a frequência da CPU e sem medir diretamente as falhas de cache. Por isso, diferenças muito pequenas podem refletir o próprio custo de medição. As contagens de operações foram coletadas uma única vez, e as implementações encadeadas de fila e pilha não foram comparadas com versões baseadas em vetores. Além disso, os tempos não incluem a leitura e a impressão dos comandos, e a memória estimada considera espaços reservados que podem permanecer alocados após remoções.

A reprodução dos experimentos depende das mesmas configurações, sementes, versões dos módulos e opções de compilação. Os arquivos CSV preservam as medidas coletadas e permitem conferir os valores apresentados nas tabelas. Mudanças nos algoritmos de busca, nas estruturas de arma-

zenamento ou na manutenção dos índices podem alterar o desempenho e exigem novas medições das operações afetadas.

## 6 Conclusão

O sistema integra um catálogo esparso migrável, fila FIFO global, pilhas LIFO por aventureiro e recomendação por ordenações globais ponderadas. A separação em TADs permite trocar a representação e testar estratégias sem mudar a semântica dos comandos. A consulta LF demonstra que valores zero que não ocupam células exigem informação sobre todos os itens cadastrados, mesmo quando nenhuma célula os representa.

Os experimentos favoreceram L nos tempos dos perfis avaliados e C na memória estimada. Manter a build atualizada a cada alteração reduziu o tempo da consulta LH, mas aumentou o custo de alterações; o ponto de compensação depende da carga. Reutilizar ordenações beneficiou Inserção nas métricas, sem garantir o menor tempo total da consulta Q. O produto intercalado venceu nas linhas mais esparsas e o denso nas mais preenchidas. O principal aprendizado é que limites assintóticos, padrão de acesso e organização efetiva do código devem ser analisados em conjunto.

Como continuidade, destacam-se reservar cabeças e usar caudas ao construir L, calcular a norma da build uma única vez por Q e avaliar ordenação híbrida. Essas propostas decorrem dos custos identificados e não são apresentadas como melhorias já implementadas ou medidas.

## Agradecimentos e Declaração de Uso de IA Generativa

Durante o desenvolvimento deste trabalho, utilizou-se IA para auxiliar na revisão de trechos de código, visando à correção de sintaxe, à identificação de possíveis vazamentos de memória e ao uso de boas práticas de programação. A IA também foi utilizada para auxiliar na análise experimental e na revisão e formatação da documentação, buscando reduzir redundâncias e melhorar a clareza do texto.

## Referências

Cormen, T. H.; Leiserson, C. E.; Rivest, R. L.; and Stein, C. 2022. *Introduction to Algorithms*. MIT Press, fourth edition. ISBN 9780262046305.

Drepper, U. 2007. What Every Programmer Should Know About Memory. Relatório técnico, versão 1.0. https://www.akkadia.org/drepper/cpumemory.pdf.

Saad, Y. 2003. *Iterative Methods for Sparse Linear Systems*. Society for Industrial and Applied Mathematics, second edition.

Sedgewick, R.; and Wayne, K. 2011. *Algorithms*. Addison-Wesley, fourth edition. ISBN 9780321573513.